



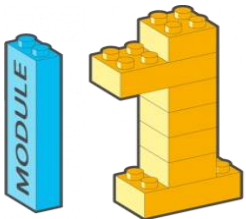
VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_PRINCIPLES OF COMPILER DESIGN



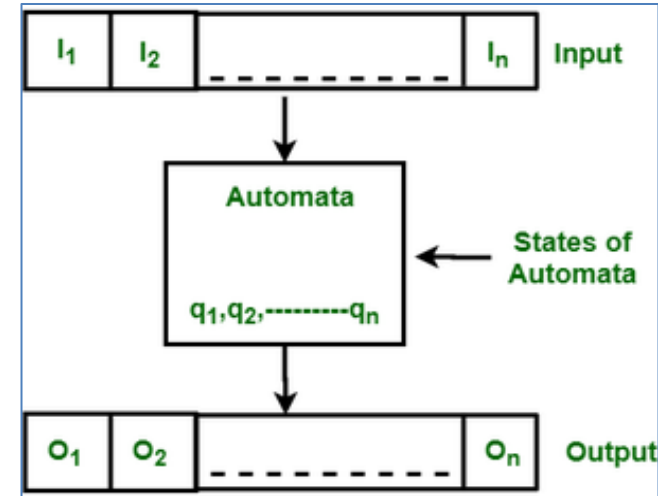
INTRODUCTION TO COMPILATION AND LEXICAL ANALYSIS



Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

Finite Automata (FA)

- Automata is the study of mathematical machinery or systems.
- Finite automata or finite state machine is the simplest machine that is used for recognizing patterns.
- The finite automata or finite state machine is an abstract machine that has five elements or tuples.
- It has a set of states and rules for moving from one state to another but it depends upon the applied input symbol.
- **Input:** All the **input strings** or **numbers** are given i.e **$I_1, I_2, I_3 \dots I_n$** .
- **Output:** Possible outputs are mentioned for the respective inputs i.e. **$O_1, O_2, O_3 \dots O_n$** .
- **Automata states:** All the states of the DFA are mentioned i.e. **$q_1, q_2, q_3 \dots q_n$** .
- **State relation:** The relationship between various states is described.
- **Output relation:** Discuss the relationship between the various output functions provided to the DFA.



- A finite automaton can be defined 5 tuple

$$(Q, \Sigma, \delta, q_0, F)$$

Q is a finite set called the set of states

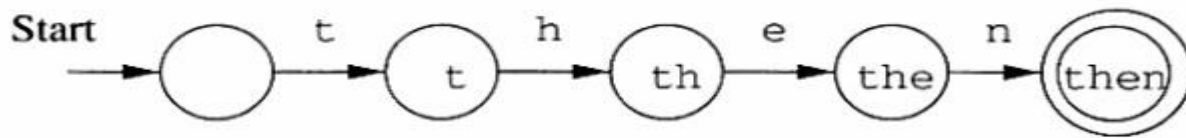
Σ is a finite set called the alphabet

δ : is the transition function and consists of transitions $Q \times \Sigma \rightarrow Q$

$q_0 \in Q$ is the initial state

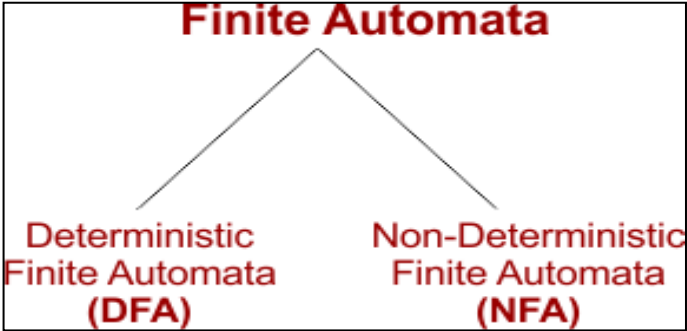
$F \subseteq Q$ is the set of final states.

- The machine always recognizes only one language but it can accept many strings.
- The machine recognizes a language even when it is not accepting a string, which is known as the empty language $\emptyset$.

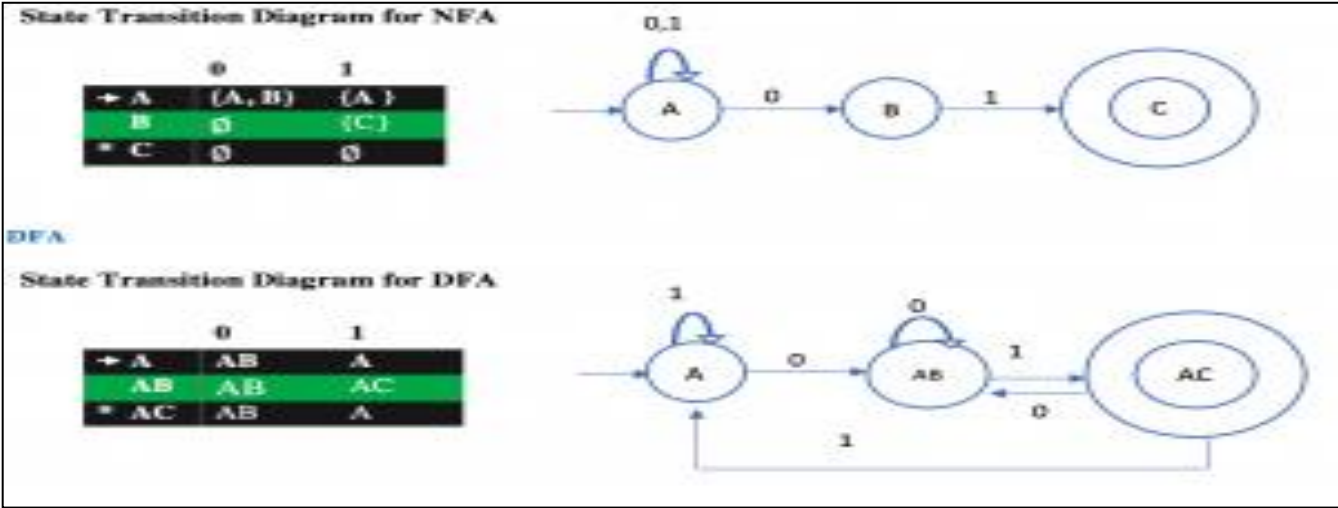


A finite automaton modeling recognition of **then**

Types of Finite Automata

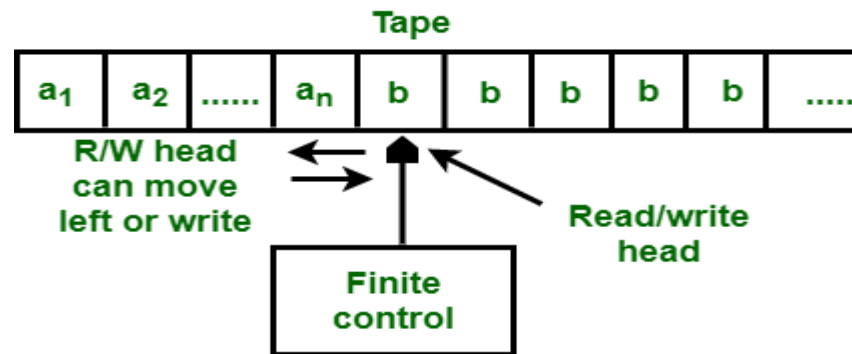


DFA	NDFA
The transition from a state is to a single particular next state for each input symbol. Hence it is called <i>deterministic</i> .	The transition from a state can be to multiple next states for each input symbol. Hence it is called <i>non-deterministic</i> .
Empty string transitions are not seen in DFA.	NDFA permits empty string transitions.
Backtracking is allowed in DFA	In NDFA, backtracking is not always possible.
Requires more space.	Requires less space.
A string is accepted by a DFA, if it transits to a final state.	A string is accepted by a NDFA, if at least one of all possible transitions ends in a final state.



Deterministic Finite Automata

- Deterministic refers to the **uniqueness** of the computation.
- The finite automata are called deterministic finite automata if the machine **read an input string one symbol at a time**.
- It is a finite state machine that either accepts or rejects strings of symbols, and for each input string it returns a unique computation.
- It has a set of states and rules for going from one state to another, depending on the input symbol.
- DFA is not capable of using an Empty String transition.
- This machine can process an input string (shown on a tape) from left to right.



- A deterministic finite automaton (in short DFA) is a 5 tuple

$$(Q, \Sigma, \delta, q_0, F)$$

Q is a finite set called the set of states

Σ is a finite set called the alphabet

δ : is the transition function and consists of transitions $Q \times \Sigma \rightarrow Q$

$q_0 \in Q$ is the initial state

$F \subseteq Q$ is the set of final states.

δ means : for every input, there will be a unique state

$$Q(state) \times \Sigma(input) \rightarrow Q(state)$$

Representation of DFA:

- A DFA can be represented by the following ways
 1. Transaction diagram DFA or State diagram
 2. Transaction table DFA
 3. Language of DFA

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

$$Q = \{a, b, c\},$$

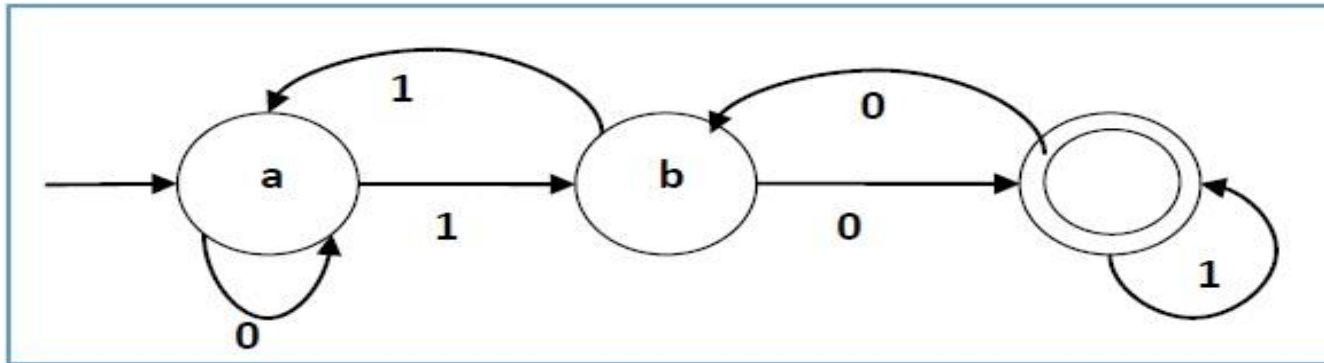
$$\Sigma = \{0, 1\},$$

$$q_0 = \{a\},$$

$$F = \{c\},$$

1. Transaction diagram

- Transaction diagram is the graphical representation of a DFA.
- It is also called state transaction diagram.
 - **Nodes** or **vertices** of the digraph represent the **states** that belong to set of states Q .
 - The connected **arcs** or **arrow labeled** with an input alphabets from Σ that show the **transition**.
 - The **initial state** is denoted by a circle with an **empty single incoming arc**.
 - The **final state** is indicated by **double circles**.



2. Transition table

- The transition table is a table that shows how the transition function works.
- It takes **two inputs** (a state and a symbol) and **outputs** a state (the "next state").
- The following elements make up a transition table:
 - **Input symbols** are represented by **columns**.
 - **States** are represented by **rows**.
 - The **next state** corresponds to the **entries**.
 - An **arrow** with no source denotes the **start state**.
 - A **star** indicates the **accepted state**.

3. Language

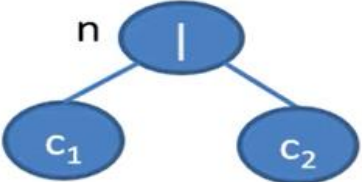
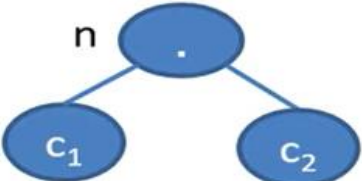
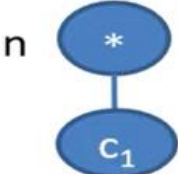
- The language of DFA consist a set of all strings chosen from Σ^* that all are accepted by machine M.

Present State (q)	Next State for Input 0 (a)	Next State for Input 1 ($\delta(q, a)$)
$\rightarrow a$	a	b
b	c	a
*c	b	c

Conversion from regular expression to DFA

Rules to compute nullable, firstpos, lastpos

- `nullable( $n$ )`
 - The subtree at node n generates languages including the empty string.
- `firstpos( $n$ )`
 - The set of positions that can match the first symbol of a string generated by the subtree at node n .
- `lastpos( $n$ )`
 - The set of positions that can match the last symbol of a string generated by the subtree at node n .
- `followpos( $i$ )`
 - The set of positions that can follow position i in the tree.

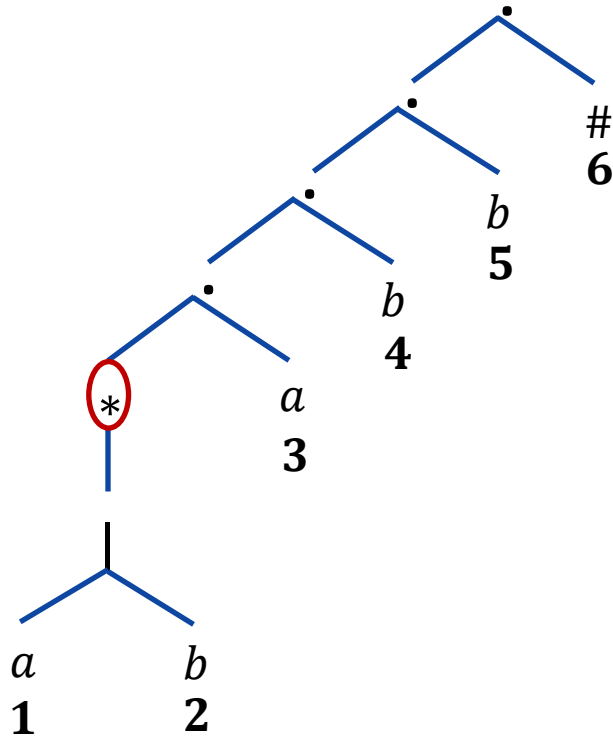
Node n	$nullable(n)$	$firstpos(n)$	$lastpos(n)$
A leaf labeled by ε	true	$\emptyset$	$\emptyset$
A leaf with position i	false	$\{i\}$	$\{i\}$
 Diagram: A node n (blue oval) containing a vertical bar $$. It has two children, c_1 and c_2 (blue ovals), connected by lines.	$nullable(c_1)$ or $nullable(c_2)$	$firstpos(c_1)$ $\cup$ $firstpos(c_2)$	$lastpos(c_1)$ $\cup$ $lastpos(c_2)$
 Diagram: A node n (blue oval) containing a dot $.$. It has two children, c_1 and c_2 (blue ovals), connected by lines.	$nullable(c_1)$ and $nullable(c_2)$	if ($nullable(c_1)$) then $firstpos(c_1) \cup firstpos(c_2)$ else $firstpos(c_1)$	if ($nullable(c_2)$) then $lastpos(c_1) \cup lastpos(c_2)$ else $lastpos(c_2)$
 Diagram: A node n (blue oval) containing an asterisk $*$. It has one child, c_1 (blue oval), connected by a line.	true	$firstpos(c_1)$	$lastpos(c_1)$

Rules to compute followpos

1. If n is **concatenation** node with left child $c1$ and right child $c2$ and i is a position in $\text{lastpos}(c1)$, then all position in $\text{firstpos}(c2)$ are in $\text{followpos}(i)$
2. If n is $*$ node and i is position in $\text{lastpos}(n)$, then all position in $\text{firstpos}(n)$ are in $\text{followpos}(i)$

Conversion from regular expression to DFA

$(a|b)^*abb\ #$



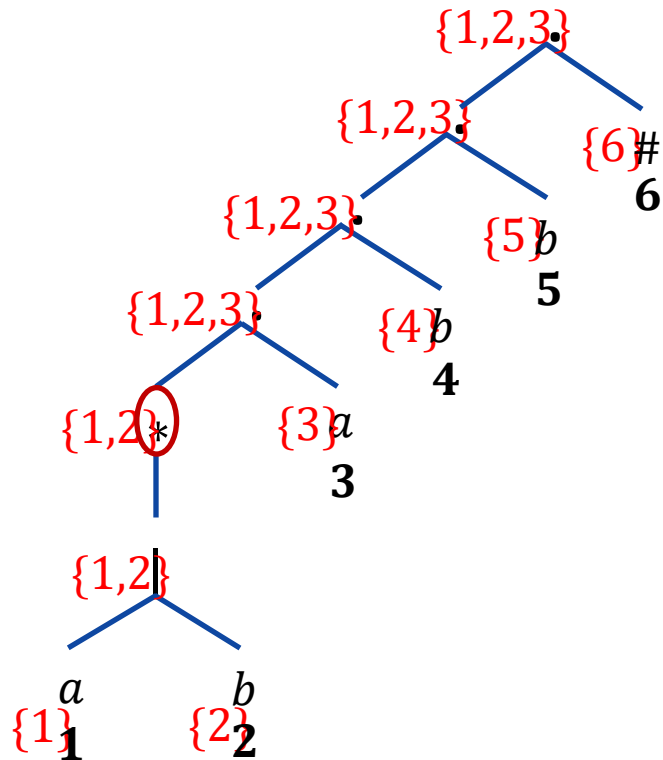
Step 1: Construct Syntax Tree

Step 2: Nullable node

Here, $*$ is only nullable node

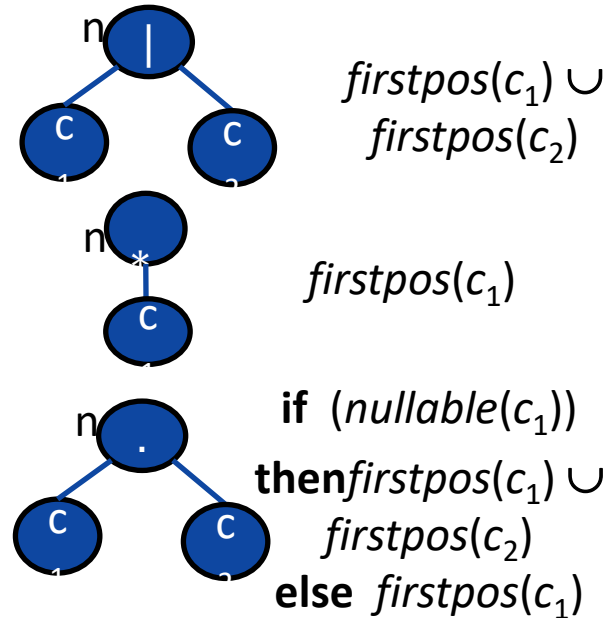
Conversion from regular expression to DFA

Step 3: Calculate firstpos



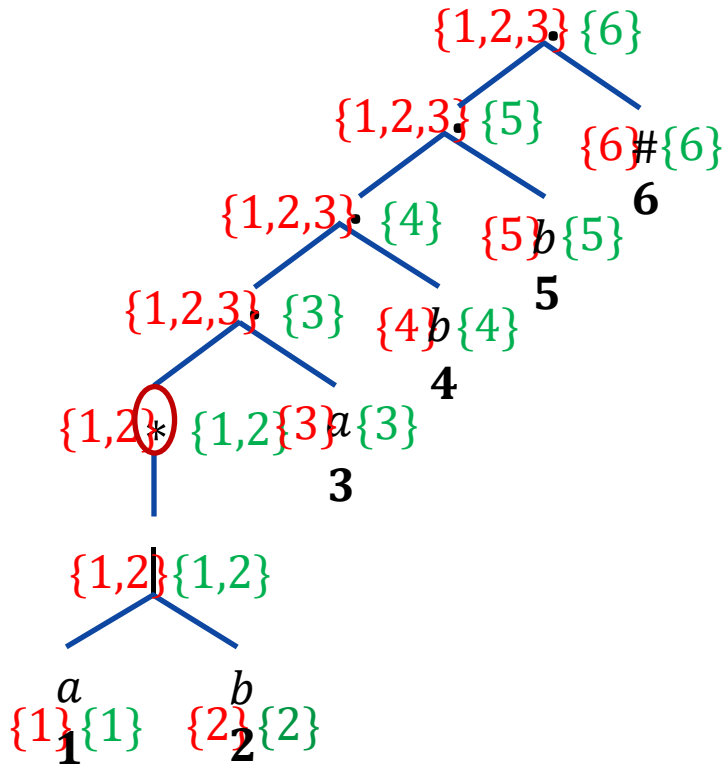
Firstpos

A leaf with position $i = \{i\}$



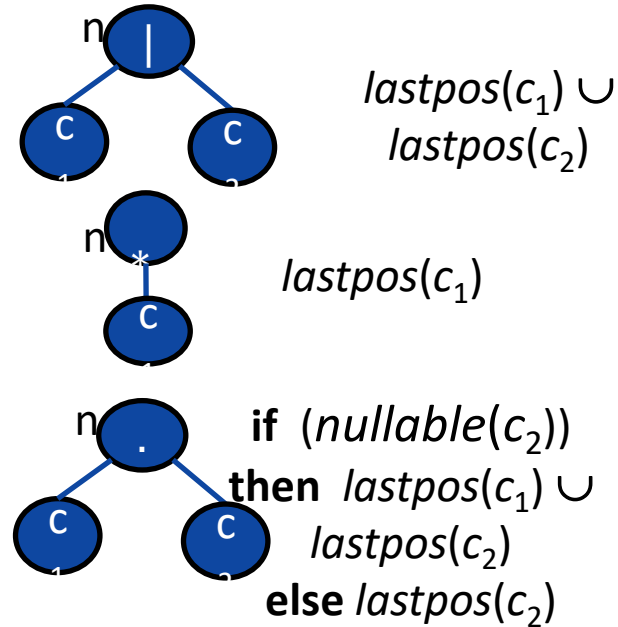
Conversion from regular expression to DFA

Step 3: Calculate lastpos



Lastpos—

A leaf with position $i = \{i\}$



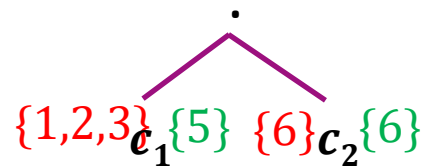
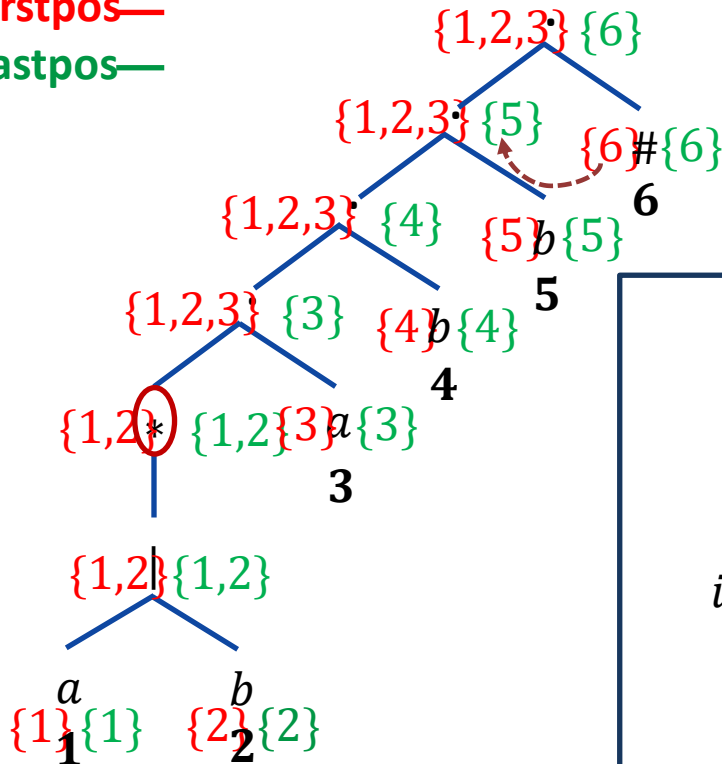
Conversion from regular expression to DFA

Step 4: Calculate followpos

Positio	followpos
9	6

Firstpos—

Lastpos—

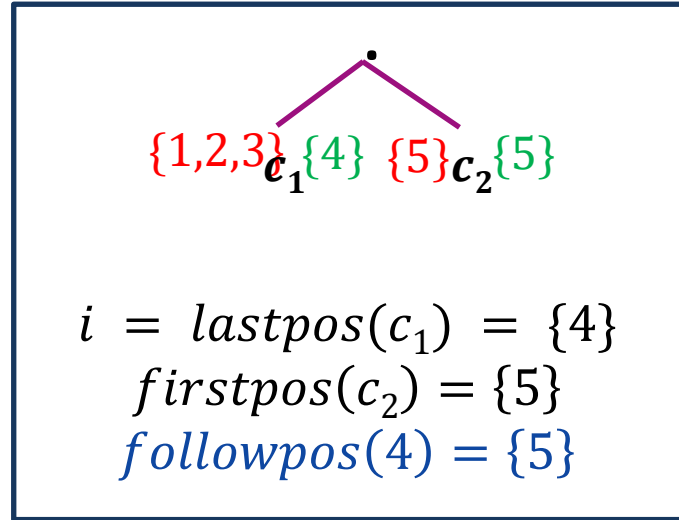
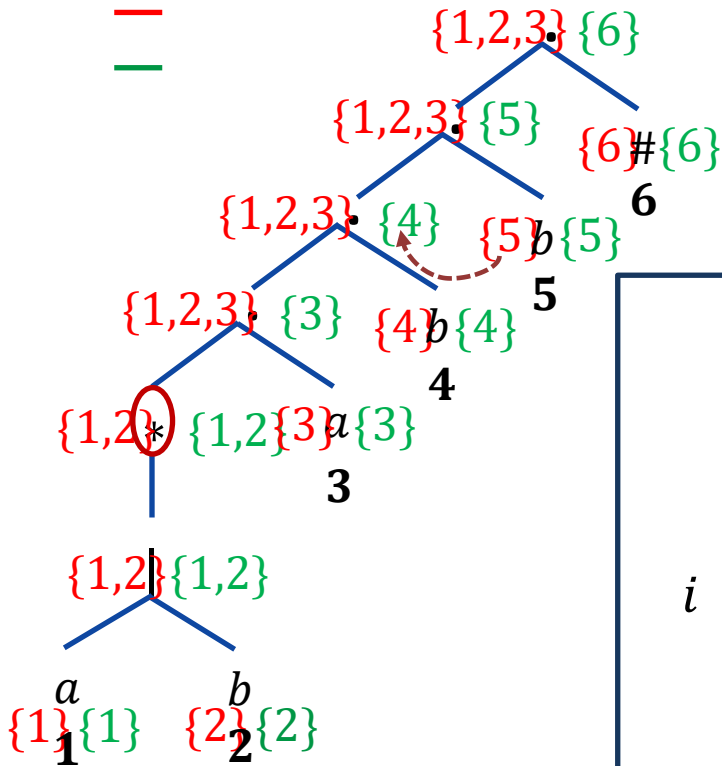


$i = \text{lastpos}(c_1) = \{5\}$
 $\text{firstpos}(c_2) = \{6\}$
 $\text{followpos}(5) = \{6\}$

Conversion from regular expression to DFA

Step 4: Calculate followpos

Positio	followpos
9	6
4	5

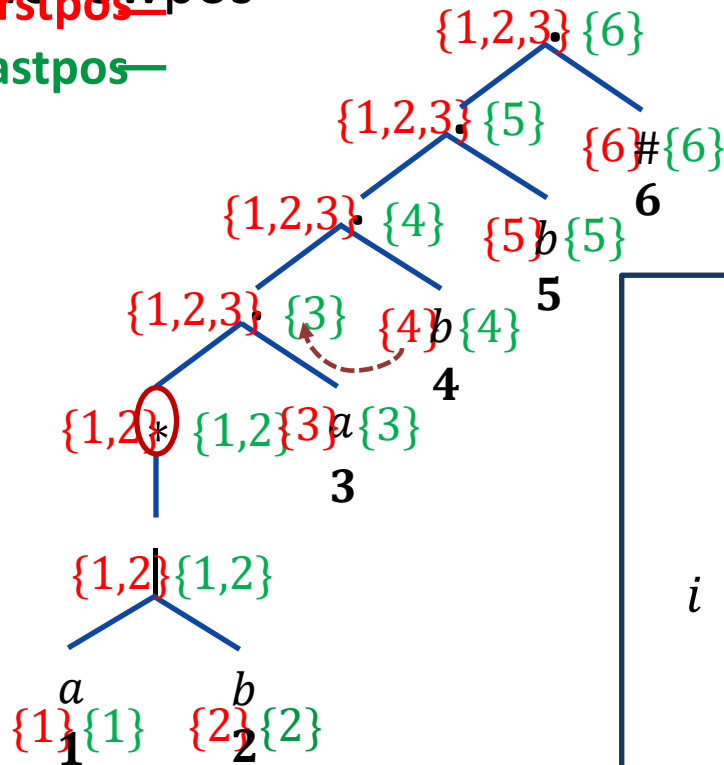


Conversion from regular expression to DFA

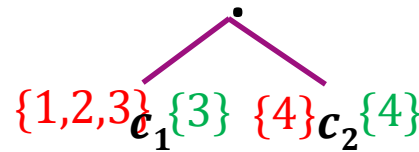
Step 4: Calculate

Firstpos —

Lastpos —



Positio	followpos
9	6
4	5
3	4



$$i = \text{lastpos}(c_1) = \{3\}$$

$$\text{firstpos}(c_2) = \{4\}$$

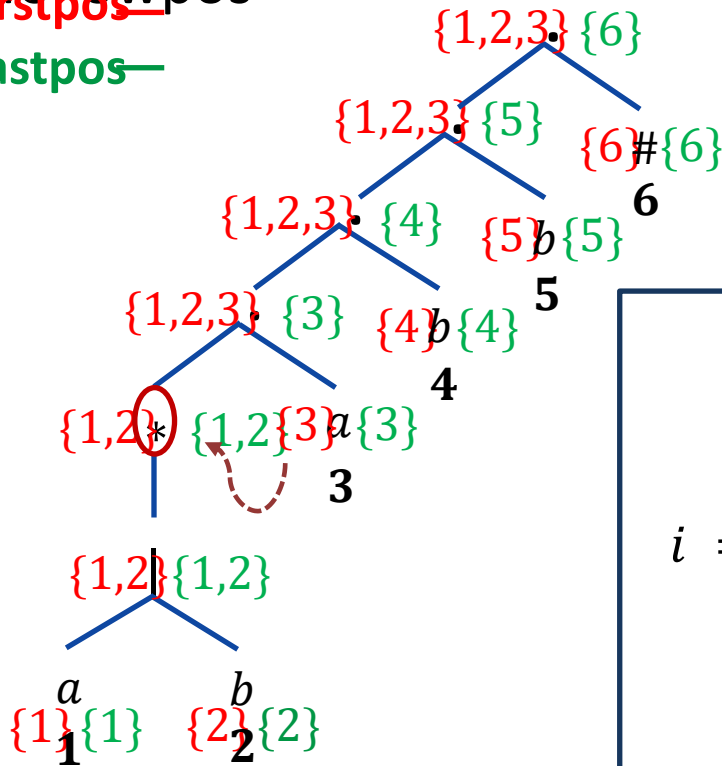
$$\text{followpos}(3) = \{4\}$$

Conversion from regular expression to DFA

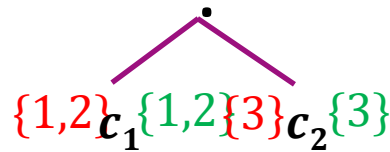
Step 4: Calculate

followpos

Lastpos—



Positio	followpos
5	6
4	5
3	4
2	3
1	3



$$i = lastpos(c_1) = \{1,2\}$$

$$firstpos(c_2) = \{3\}$$

$$followpos(1) = \{3\}$$

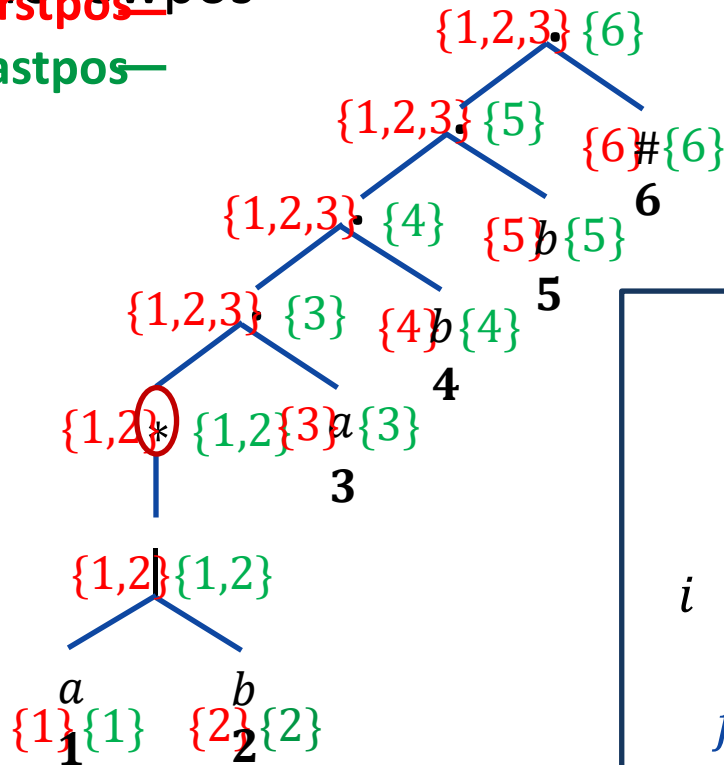
$$followpos(2) = \{3\}$$

Conversion from regular expression to DFA

Step 4: Calculate

Firstpos —

Lastpos —



Positio	followpos
6	6
4	5
3	4
2	1,2
1	1,2

$\{1,2\}^* \{1,2\}$
 n

$i = \text{lastpos}(n) = \{1,2\}$
 $\text{firstpos}(n) = \{1,2\}$
 $\text{followpos}(1) = \{1,2\}$
 $\text{followpos}(2) = \{1,2\}$

Conversion from regular expression to DFA

Initial state = *firstpos* of root = {1,2,3} -----

A

State A

$\delta((1,2,3),a) = \text{followpos}(1) \cup \text{followpos}(3)$
 $= (1,2,3) \cup (4) = \{1,2,3,4\}$ ----- B

$\delta((1,2,3),b) = \text{followpos}(2)$
 $= (1,2,3)$ ----- A

Positio	followpos
9	6
4	5
3	4
2	1,2,3
1	1,2,3

States	a	b
A={1,2,3}		
B={1,2,3,4 }		

Conversion from regular expression to DFA

State B

$$\begin{aligned}\delta((1,2,3,4),a) &= \text{followpos}(1) \cup \text{followpos}(3) \\ &= (1,2,3) \cup (4) = \{1,2,3,4\} \text{ ----- B}\end{aligned}$$

$$\begin{aligned}\delta((1,2,3,4),b) &= \text{followpos}(2) \cup \text{followpos}(4) \\ &= (1,2,3) \cup (5) = \{1,2,3,5\} \text{ ----- C}\end{aligned}$$

Positio	followpos
9	6
4	5
3	4
2	1,2,3
1	1,2,3

State C

$$\begin{aligned}\delta((1,2,3,5),a) &= \text{followpos}(1) \cup \text{followpos}(3) \\ &= (1,2,3) \cup (4) = \{1,2,3,4\} \text{ ----- B}\end{aligned}$$

$$\begin{aligned}\delta((1,2,3,5),b) &= \text{followpos}(2) \cup \text{followpos}(5) \\ &= (1,2,3) \cup (6) = \{1,2,3,6\} \text{ ----- D}\end{aligned}$$

States	a	b
A={1,2,3}	B	A
B={1,2,3,4}		-
C={1,2,3,5}		
D={1,2,3,6}		
}		

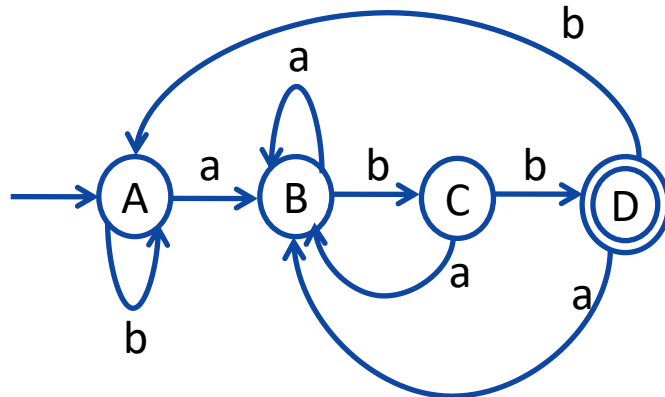
Conversion from regular expression to DFA

State D

$$\begin{aligned}\delta((1,2,3,6),a) &= \text{followpos}(1) \cup \text{followpos}(3) \\ &= (1,2,3) \cup (4) = \{1,2,3,4\} \text{ ----- B}\end{aligned}$$

$$\begin{aligned}\delta((1,2,3,6),b) &= \text{followpos}(2) \\ &= (1,2,3) \text{ ----- A}\end{aligned}$$

Positio	followpos
6	6
4	5
3	4
2	1,2,3
1	1,2,3



DFA

States	a	b
A={1,2,3}	B	A
B={1,2,3,4}	B	C
C={1,2,3,5}	B	D
D={1,2,3,6}		
}		